

2-HOUR CRASH PREP

Aishwarya Sharma · Senior Adobe Commerce (Magento) Consultant · Infosys, Bengaluru

9+ years Magento · Adobe Certified 2023 · Lead Developer @ Altimetrik

Your 2-Hour Battle Plan (follow the clock)

Time	Do this	Why
0:00 – 0:15	Read pages 1–3: intro script + why-switch + break answer. Say each OUT LOUD once.	These decide the first impression and the only tricky questions.
0:15 – 0:50	Pages 4–6: technical rapid-fire. Read aloud the bold openers of each answer.	Covers ~90% of what Round 1 asks.
0:50 – 1:15	Page 7: consultant scenarios — memorise the FRAMEWORKS, not words.	This is what makes you 'Consultant', not 'Developer'.
1:15 – 1:35	Page 8: your 4 STAR stories. Speak each in under 2 minutes.	Every behavioral question maps to one of these.
1:35 – 1:45	Page 9: salary numbers + HR lines. Fix your numbers NOW.	Never decide numbers live on the call.
1:45 – 2:00	STOP READING. Water, dress, breathe. Say the intro once in the mirror.	A calm brain beats a crammed one.

HEADLINES

Three messages to repeat in every round: 1) **Nine years, one platform** — I'm a Magento specialist, not a generalist. 2) **I've done the whole lifecycle** — requirements to production support. 3) **I lead with quality** — certified, code-review and SonarQube driver, mentor.

DELIVERY

Golden rules: Pause 2 seconds before answering · Answer in 30–60 sec then STOP · Say “I haven't used that in production, but here's my understanding...” instead of bluffing · Before big scenarios ask: “Is this B2C or B2B? Roughly how many SKUs?”

1 · Your 90-Second Opener — “Tell me about yourself”

SAY THIS

“Thank you. I'm Aishwarya Sharma — I've spent the last **nine years on one platform: Magento, now Adobe Commerce**. I'm an **Adobe Certified Developer**, and currently a **Lead Developer at Altimetrik**, where I lead the Magento workstream for a European fashion-retail client — custom module development, **Adyen payment integration**, and I drive our code-quality program with **SonarQube** and production monitoring with **New Relic**.

Before that I was at **Xperi**, where I mentored junior developers and modernised legacy code. Earlier, at Neosoft and Navabrind, I delivered for brands like **Decathlon and FNAC** — custom REST APIs, a **two-way ERP sync module**, Multi-Source Inventory, a recommendation engine, and a full **version upgrade with Stripe integration**. I started my career building a personalised-gifting platform, integrating payments, shipping and notifications end-to-end.

So across nine years I've covered the full lifecycle — requirements, estimation, custom development, integrations, performance, releases, production support — and increasingly, **leading others while doing it**. That's exactly why this Senior Consultant role excites me. Happy to go deeper into any part.”

The last line makes them ask about your CURRENT project — your home ground. That's deliberate.

2 · “Why do you want to switch?” (your only tricky topic — own it)

SAY THIS

“Altimetrik has been great — I lead the Magento workstream, drive quality, mentor the team. So I'm not running away from anything. I'm applying because of what this role **is**: end-to-end Adobe Commerce consulting — discovery, solutioning, architecture, delivery. After nine years deep on this platform, the consultant track is my natural next step, and Infosys offers it at enterprise scale on exactly my specialisation. That's why this role, and why now.”

If pushed: “You joined Altimetrik recently — will you leave us too?”

SAY THIS

“Fair question, I'll be transparent. I took a planned maternity break, and Altimetrik was the right **re-entry** — I'm grateful, and I've delivered fully: I took lead responsibility within months. But that move was about the right re-entry; **this move is about the right long-term platform** for my specialisation. I'm being very selective — I'm only looking at senior Adobe Commerce consulting roles. When I move, I intend it to be my long-term home, growing from senior consultant toward architect — and Infosys has that ladder inside its Adobe practice.”

If asked about the 2024 gap:

SAY THIS

“Yes — that was my maternity break, completely planned. I closed out my Xperi responsibilities properly, and later in the break I stayed current with Adobe Commerce releases. I returned at Altimetrik and was leading a workstream within months, so the transition back was smooth.” *(One confident paragraph. Then move on. Do not apologise.)*

NEVER SAY

Anything negative about Altimetrik · salary as the main reason · “better opportunity” with no specifics · over-explaining the break · hesitation about on-site (you're in Bangalore — “completely fine, my family setup fully supports my career”).

Why Infosys? — three reasons, ready to fire:

SAY THIS

“**Scale** — global enterprise Adobe Commerce programs with complex catalogs and real integrations. **Career path** — a clear consultant-to-architect track in the digital commerce practice. **Structure** — mature delivery, learning culture, long-running clients. I know the environment I do my best work in, and this is it.”

3 · Technical Rapid-Fire (Round 1 core — read openers aloud)

Plugins vs Observers vs Preferences (asked in EVERY senior interview)

SAY THIS

“My order of preference: **plugin first**, **observer for event-driven side-effects**, **preference as last resort**. A plugin intercepts a public method — **before** modifies inputs, **after** modifies the result, **around** only when I must control whether the original runs, because around hurts performance. Declared in di.xml with sortOrder; multiple modules can coexist — most upgrade-safe. An **observer** reacts to an event (order placed → notify ERP) via events.xml without changing behaviour. A **preference** replaces a class globally — only one wins, breaks on upgrades, so only when there's no method to intercept and no event.” *Follow-up: no plugins on private/final methods or final classes. Order: all before (by sortOrder), arounds wrap like an onion, then all after.*

Dependency Injection · Factory · Proxy · Virtual Type

SAY THIS

“Constructor injection configured in di.xml — type-hint interfaces, Object Manager injects implementations; we never call ObjectManager directly. A **Factory** (auto-generated) gives NEW instances at runtime. A **Proxy** lazy-loads expensive dependencies — classic fix for slow CLI commands. A **Virtual Type** is a di.xml-only configured variant of a class — no new PHP class needed.”

Service Contracts

SAY THIS

“The module's public API — Api/ interfaces like ProductRepositoryInterface plus data interfaces. Customisations and web APIs go through contracts, never concrete models — Adobe keeps them backward-compatible, so code survives upgrades. webapi.xml exposes them as REST automatically. In reviews I flag direct model access where a repository exists.”

EAV in one breath

SAY THIS

“Entity–Attribute–Value: products/categories/customers store attributes in typed value tables (varchar, int, decimal, datetime, text) — flexible: merchants add attributes without schema changes, and values vary per store view (multi-language). Cost = heavy joins, compensated by indexers and Elasticsearch/OpenSearch for listings. Orders and quotes are flat tables. Attribute scopes: global / website / store-view — price is website scope by default.”

Place Order → what actually happens (draw it if offered a whiteboard)

SAY THIS

“Cart = **quote** (quote, quote_item, quote_address, quote_payment). Place Order → payment method → quote converts via sales_convert_quote mapping into **sales_order** + items/address/payment, firing sales_order_place_after. **Invoice** = the money (online capture for Adyen/Stripe or offline). **Shipment** = the goods + tracking. **Credit memo** = refund. State (new/processing/complete/closed/canceled/holded) is the internal machine; statuses are labels mapped to states. With **MSI**, ordering creates a **reservation** — logical deduction; physical deduction happens at shipment.”

Catalog vs Cart price rules

SAY THIS

“Catalog rules change the displayed price itself, applied by the price indexer — 'flat 20% off category X'. Cart rules apply during totals collection — coupons, buy-X-get-Y, free shipping. Totals pipeline: subtotal → discount → shipping → tax → grand total; custom fees = new total collector in sales.xml. I maintained catalog pricing rules for promotions on Surfline.”

Caching — the answer that impresses (FPC + private content)

SAY THIS

“Production full-page cache should be **Varnish** — Fastly on Adobe Commerce Cloud. Per-user parts (cart, name) are solved by **private content**: pages cache generically, the browser fills personal blocks via the **customer-data JS** library pulling JSON 'sections' stored in localStorage; **sections.xml** maps actions to sections to invalidate (add-to-cart → cart section). **cacheable="false" on ANY block makes the whole page uncacheable** — a real bug I've fixed. Below FPC: Redis for cache backend + sessions; cache **tags** give precise invalidation — saving a product purges only its pages.”

Indexers & 'Update by Schedule'

SAY THIS

“Indexers pre-compute price, catalog rules, category-product, stock, search. **Update on Save** reindexes synchronously — fine for tiny catalogs. **Update by Schedule** uses **mview**: MySQL triggers write changed IDs into `_cl` changelog tables and cron consumes deltas. Mandatory at scale — e.g. frequent ERP price feeds. Ops: indexer:status, resetting a stuck indexer, watching changelog backlog.”

“A production site is slow — triage it” (STAR question, rehearse!)

SAY THIS

“First split it: everyone or some users? All pages or one flow? Since when — correlate with the last release. Then layers: **Edge** — is Varnish/Fastly actually HITting or did a cookie/cacheable=false regression kill it? **App** — New Relic, which I use daily: slowest transactions, worst queries, blocking external calls. **DB** — slow query log, missing indexes, imports in business hours. **Search** — ES heap. **Infra** — PHP-FPM saturation, Redis eviction. Usual suspects I've actually fixed: uncacheable blocks, N+1 model loads in loops, Update-on-Save at scale, cron pileups, inline third-party HTTP calls. Fix, measure, then add an alert so it can't return silently.”

APIs: REST vs GraphQL vs Bulk

SAY THIS

“**REST** for system-to-system (ERP, OMS) — `webapi.xml` routes to service contracts with ACL; I built custom REST modules on Peelworks. **GraphQL** for storefront/headless — exact fields, one round trip, PWA Studio's API. **Async/Bulk REST** queues into RabbitMQ for volume — right pattern for thousands of ERP price/stock updates. Auth: integration tokens server-to-server, OAuth for apps, customer/admin tokens user-scoped.”

Payment integration (you own this topic — Stripe, Adyen, PayUbiz, Paytm)

SAY THIS

“Gateway framework: command pool (authorize/capture/void/refund), request builders, HTTP client, response handlers/validators; frontend KO component tokenises card data client-side — that's the PCI boundary. The underestimated parts I design for: **webhooks handled idempotently** — Adyen is notification-driven, order state settles async — plus edge states (captured but order save failed, duplicate notifications, partial refunds) and reconciliation against gateway reports. I've debugged Adyen notification-cron failures in production.”

Adobe Commerce vs Open Source + Cloud (one-liner each)

SAY THIS

“Commerce adds B2B suite, Content Staging, customer segments, RMA, gift cards, Sensei Product Recommendations / Live Search, BI — and on Cloud: managed PaaS with Integration/Staging/Production, git + ece-tools deployments (.magento.app.yaml), Fastly CDN/WAF, New Relic baked in. My hosting hands-on maps one-to-one: Varnish→Fastly, Redis, Elasticsearch, deploy scripts.” *If probed on Cloud production experience — be honest, say exactly that mapping sentence.*

Upgrade playbook (you did 2.1.18 → 2.3.5 for real)

SAY THIS

“Audit third-party modules FIRST — that’s 80% of upgrade pain. Branch: composer bump, fix DI compile + deprecated APIs, patch each extension. Staging with production data: checkout, payments sandbox, integrations, perf smoke. Release with rollback point. I did exactly this on Tivoliwines with Stripe added on top.”

Security quick hits

SAY THIS

“Prompt security patches (security-only line), escapeHtml/escapeUrl vs XSS, form keys vs CSRF, ACL on every admin route, prepared statements, locked admin URL + built-in 2FA, least-privilege tokens, reCAPTCHA, WAF/Fastly. Code review is my last gate — I specifically look for unescaped output and missing ACL.”

4 · Consultant Scenarios — memorise FRAMEWORKS, not words

Rule: clarify → structure → options with trade-offs → ONE recommendation.

Scenario	Your framework (say the bold spine)
New store: discovery & estimation	4 buckets: Business (B2B/B2C, markets, volumes) · Catalog (SKUs, complexity, PIM?) · Integrations (ERP, payments, tax, carriers, CRM) · Non-functional (perf, compliance, dates). Decompose into epics, three-point estimates, risk buffer on integrations & migration. Phase it: MVP launch + fast-follow. Estimates carry written assumptions.
Checkout conversion dropped after release	Incident, not debate. Quantify which funnel step from analytics → cross-check New Relic + JS errors (often one browser / payment method / guest segment) → diff the release → check the PSP dashboard before blaming code → mitigate first (rollback / feature-flag) → postmortem + synthetic checkout monitor.
Multi-country rollout (5 countries)	Website per country (own currency, tax, payments) · store views per language · price scope per website. Surface: single vs regional instances, local payment methods, hreflang/domains, content-ops via Content Staging. Pilot one country, template, stamp out the rest.
1M SKUs nightly from PIM	Push back: deltas, not full loads (full = recovery only). Async bulk API / queue consumers, batched writes, reindex after batch, tag-based cache invalidation, media to CDN async. Guardrails: validation quarantine, duration alerts, reconciliation count. Test at production scale on staging.
Define team standards (JD bullet!)	I do this today: Automated (SonarQube gates + Magento PHPCS in CI — I drove adoption and burned the backlog down) · Process (review checklist: contracts over models, plugin over preference, ACL, escaping) · Human (mentor, don't gatekeep — standards stick when the team knows the why).
Vendor module conflicts with our code	Diagnose with dev:di:info (plugin list, sortOrder, preference winner) → short-term: adjust sortOrder/scope → right fix: vendor ticket + composer patch tracked in git, re-validated each upgrade. Never silently fork a vendor module.
Production down on sale day	Roles first: one person communicates (update every 30 min), I drive diagnosis. Check fast: last deploy? traffic? infra? PSP down? Restore before root-cause: rollback / scale / serve cached / degrade gracefully — checkout availability is the only KPI. Blameless postmortem; every incident buys a permanent alert or runbook line.

5 · Your 4 Stories (2 minutes each, out loud once)

Story	Spine (S→T→A→R)
1 · Lead & quality driver — Altimetrik Use for: leadership, initiative, current work	Lead Magento for EU fashion client; inherited heavy tech-debt + production noise (Adyen cron errors in New Relic). Owned quality AND delivery: triaged SonarQube backlog, assigned across team, fixed hardest categories myself, resolved Adyen notification failures, made review standards explicit. Result: backlog near-zero, error noise visibly down, team now holds the quality gate without me policing.
2 · Complex integration — Peelworks ERP Use for: hardest technical problem, integrations	Magento 2 EE backend for Android commerce app + ERP. Built custom REST APIs (order/product tracking) and two-way ERP sync module: source of truth per entity (ERP: catalog/stock · Magento: orders), delta updates, idempotent retries, MSI reservations. Result: reliable consistency; this became my reusable integration pattern — exactly what this role needs.
3 · Upgrade & payments under pressure — Tivoliwines Use for: risk management, ownership	Live UK wine store on unsupported 2.1.18. Owned upgrade to 2.3.5 + Stripe. Audited every extension FIRST, branch upgrade, fixed deprecated APIs, staging validation with sandbox payments, planned cutover with rollback point; Stripe via gateway framework with webhooks. Result: clean cutover, zero order loss, my permanent upgrade playbook.
4 · Mentoring & standards — Xperi Use for: mentorship, teamwork	Growing team, slow ramp-up on legacy code. Documented workflows for new hires, ran best-practice sessions (plugins over preferences, contracts, layout), teaching-style reviews, modernised legacy modules as clean examples. Result: faster onboarding, shorter review cycles — mentoring is now core to how I work.

Quick behavioral answers

Question	Anchor
Strength?	"Depth plus ownership — nine years on one platform, and I take things end-to-end."
Weakness?	"I used to hold work too close — I'd rather fix it myself than delegate. Leading at Altimetrik forced me to delegate with clear standards; mentoring made me better at it. I still catch myself on critical fixes."
A failure?	"Early on I underestimated third-party module risk in an upgrade — cost extra testing days. Now extension audit is step one of my playbook." (Ends as growth.)
Unrealistic client demand?	"I never give a flat no — I show options with costs: X by the date if we defer Y, or full scope two weeks later. Clients respect trade-offs, not refusals."
Tight deadline?	"Scope negotiation beats heroics — must-have vs fast-follow, protect quality on the critical path. A broken checkout shipped on time is still a failure."

6 · HR & Salary — decide numbers NOW, not on the call

Item	Your position
Current CTC	Rs 27 LPA — state it plainly. It's a strong number.
Opening ask	Rs 38–40 LPA — “based on the seniority of the role and my specialisation.”
Landing zone	Rs 34–36 LPA (30%+ hike). Below 33 the move stops making sense — be ready to hold.
Notice period	Know your exact Altimetrik notice. “I'll resign the day I have the offer letter and pursue early release.”
Other offers?	“I'm in active discussions elsewhere, but this role is my clear first preference — it's exactly my specialisation.” (Never fabricate a specific offer.)

The salary answer, verbatim:

SAY THIS

“My current package is 27 lakhs. Given this is a senior consultant position and my nine years of specialised Adobe Commerce experience map directly onto every line of the role, I'm looking at around 38 to 40 lakhs. That said, role fit matters most — if we're aligned on the role and level, I'm confident we can align on numbers.” (Say it once. Then *SILENCE* — *silence is negotiation*.) If countered low: “Given the market for certified senior Magento consultants, and that I'd come in at the top of the experience band, is there flexibility toward 38? I'm open to a joining bonus if the fixed band is constrained.”

NEVER SAY

“Negotiable as per company norms” · a range starting at your minimum · justifying by expenses. Justify only by market value and role scope. Negotiate **FIXED** before variable.

Ask THEM (pick 2–3 — asking nothing reads as low interest):

Question	Signals
“What does the Adobe Commerce practice look like — team size, one large account or across clients?”	Practice thinking
“What does success look like for this role in the first six months?”	Outcome orientation
“Is there a path toward the Adobe Commerce Architect certification within the practice?”	Ambition + retention
“How are solution decisions made with clients — does the consultant own the technical recommendation?”	Consultant mindset

7 · Final 15 Minutes Before the Call

✓	Water on the desk · notebook + pen (write down multi-part questions)
✓	If video: camera at eye level, light on your face, join 5 minutes early
✓	Say the 90-second intro ONCE in the mirror — then stop rehearsing
✓	Three slow breaths. Sit forward. Smile when you greet.

Nine years. One platform. You have already done this job — the interview is just you describing your own work, calmly.

Walk in like you own it. Because you do. ■